UE24CS352A MACHINE LEARNING · MINI-PROJECT

Finding ships in satellite photos — explained from first principles

Team 9 · Project 35 · Rohan Suresh (PES1UG24CS383) & Satvik L (PES1UG24CS423)

A beginner-friendly walk through every idea in our project: what the data is, what each model does, why we made each choice, and what our numbers mean.

Contents

0 · How to use this guide

1 · The problem

2 · What is machine learning?

3 · What is an image to a computer?

4 · Our dataset

5 · Train, dev and test

6 · Feature engineering

7 · The five classical models

8 · Measuring how good a model is

9 · Neural networks from zero

10 · Convolutional neural networks

11 · Fighting overfitting

12 · Transfer learning

13 · Our results, explained

14 · Us vs. the Stanford paper

15 · The code, file by file

16 · The live demo

17 · Test yourself

18 · Glossary

0 How to use this guide

This guide assumes you know *nothing* about machine learning. Every term is explained the first time it appears, and each chapter builds on the ones before it. If you read it top to bottom you will understand every line of our project and be able to answer questions about it.

Throughout, you'll see coloured boxes:

IN PLAIN WORDS
A simple, everyday explanation of an idea, with no maths.

WORKED EXAMPLE
The idea applied to small, concrete numbers you can check by hand.

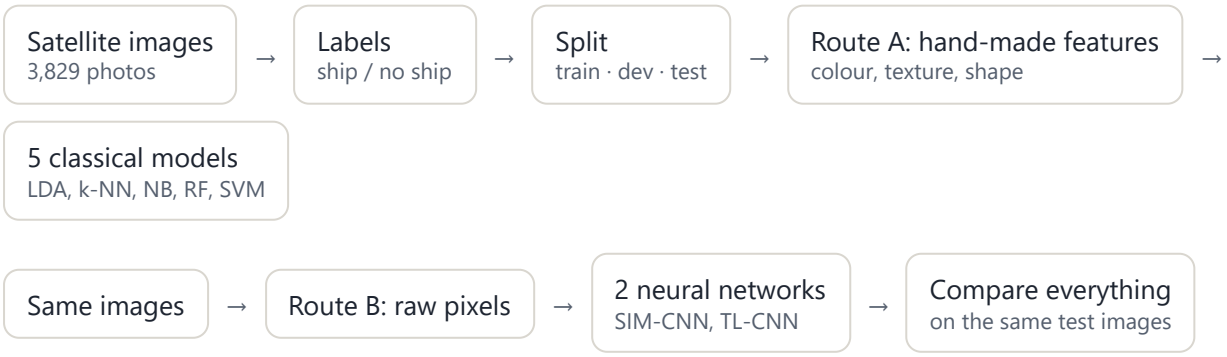
IN OUR PROJECT
Exactly how the idea shows up in our code and results.

VIVA TIP
How to explain this to the panel, or a question they are likely to ask.

COMMON MISTAKE
A misunderstanding worth avoiding.

There are also **interactive demos** — sliders and buttons you can play with. They run entirely inside this file; no internet is needed.

The whole project on one page



In one sentence: **we taught computers to look at a satellite photo and say whether there is a ship in it, using two very different approaches, and measured which works better.**

1 The problem

Why would anyone want this?

The oceans are huge, and lots happens on them that people need to know about: illegal fishing, piracy, smuggling, oil spills, ships in distress, busy shipping lanes. Satellites photograph the whole Earth constantly, but there are far too many photos for humans to look through. If a computer could flag "this photo contains a ship", people could focus only on the interesting ones.

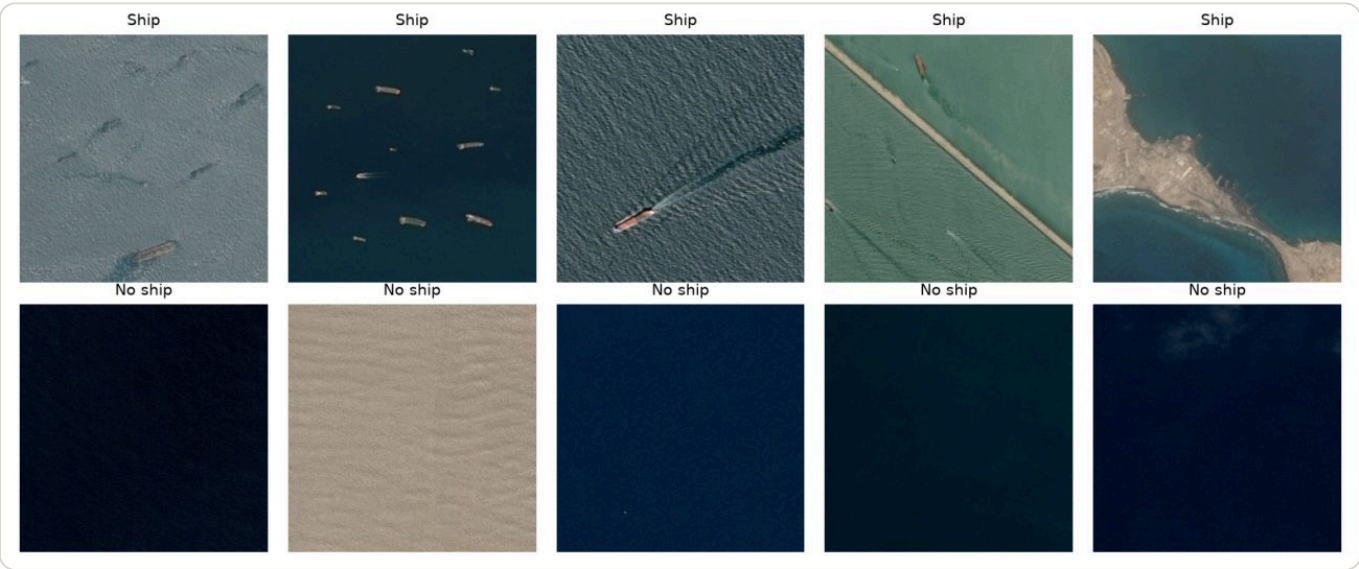
In 2018 the company **Airbus** (which operates Earth-observation satellites) ran a public competition on **Kaggle** — a website that hosts data-science competitions — called the *Airbus Ship Detection Challenge*. They released about 192,000 satellite photos with the ships marked.

A group of Stanford students (Ying Chen, Junwen Zheng and Zhengqing Zhou) used that data for their CS229 machine-learning course project. **Our project reproduces their study.**

The exact question we answer

> **IN PLAIN WORDS**
> Show the computer a satellite photo. It must answer one yes/no question: **"Is there at least one ship in this picture?"**

A task where the answer is one of a fixed set of categories is called **classification**. When there are exactly two categories (here: *Ship* and *No ship*) it is **binary classification**. Each category is called a **class**.



Real images from our dataset. Top row: *Ship*. Bottom row: *No ship*. Notice how small the ships are — often just a few pixels.

Why is this hard?

- **Ships are tiny.** In a 768×768 photo a ship might cover 50 pixels — less than 0.01% of the image.
- **Lots of things look like ships**: waves, rocks, piers, buildings on the coast, cloud edges.
- **Scenes vary enormously**: open ocean, harbours, cities, clouds, haze, different lighting and seasons.
- Some images contain many ships (up to 15 in our data), some contain one tiny one.

Classification vs. detection vs. segmentation

These three words describe increasingly detailed tasks:

Task	Question answered	Output
Classification (what we do)	Is there a ship?	Yes / No
Detection	Where are the ships?	A box around each ship
Segmentation (the Kaggle competition)	Exactly which pixels are ship?	An outline (mask) of each ship

> **VIVA TIP**
> If asked "is this really detection?": the project title says *detection* in the everyday sense of detecting whether a ship is present, but technically we do **image-level binary classification**, exactly like the Stanford paper. The original competition was segmentation; both the paper and we simplified it.

2 What is machine learning?

Rules vs. examples

The traditional way to make a computer do something is to write **rules**: "if the pixel is bright and surrounded by dark blue, it's a ship". But nobody can write rules that handle every wave, cloud and harbour. The rules would be endless and still wrong.

Machine learning (ML) flips this around: instead of writing the rules, we give the computer thousands of **examples** where we already know the answer, and an algorithm figures out its own rules from them.

> **IN PLAIN WORDS**
> Think of how a child learns what a dog is. Nobody gives them a definition like "four legs, fur, tail, barks". They see many dogs (and cats, and cows), someone says "dog" or "not a dog", and eventually they can recognise dogs they've never seen. ML works the same way.

The vocabulary you need

Example (sample, data point)
One thing we learn from. For us: one satellite image.

Label (target, ground truth)
The correct answer for an example. For us: 1 = ship, 0 = no ship.

Features
The numbers that describe an example and that the model looks at. Could be raw pixel values, or numbers we compute such as "how much blue is in the image".

Model
A mathematical function that takes features in and produces a prediction. It has adjustable internal numbers called **parameters** (or **weights**).

Training (fitting, learning)
Automatically adjusting the parameters so the model's predictions match the labels on the examples.

Prediction (inference)
Using the trained model on a new example whose label it hasn't seen.

Supervised learning
Learning from examples that come with labels — which is what we do.

The single most important idea: generalisation

A model that just memorises its training examples is useless — like a student who memorises past exam answers without understanding them. What we want is a model that **generalises**: performs well on *new*

examples it has never seen.

That's why we always test models on images that were kept hidden during training (Chapter 5).

> **COMMON MISTAKE**
> "My model gets 99% on the training data, so it's great." Not necessarily! It may have memorised. Our TL-CNN reached 98.9% on training images but 95.3% on unseen test images. Only the unseen-data number tells you how good the model really is.

Overfitting and underfitting

Overfitting = the model learned the training examples *too* specifically, including their random quirks, and does worse on new data. **Underfitting** = the model is too simple to capture the real pattern, so it does badly even on training data. A good model sits between the two. Chapter 11 covers the tools we used against overfitting.

3 What is an image to a computer?

Pixels and numbers

A digital image is a grid of tiny coloured squares called **pixels**. Our original images are 768 pixels wide and 768 tall — 589,824 pixels.

Each pixel's colour is stored as three numbers: how much **R**ed, **G**reen and **B**lue light to mix. Each number goes from 0 (none) to 255 (full). So (0,0,0) is black, (255,255,255) is white, (255,0,0) is pure red, and (20,60,110) is a dark sea blue.

So to a computer, a colour image is a block of numbers with shape **height × width × 3** (the 3 being the R, G, B **channels**). A 768×768 image is 768×768×3 = **1,769,472 numbers**.

Interactive demo — shown here in its starting state. Open Beginners_Guide.html in a browser to use it.

Try it: mix a colour

R G

B

RGB = (20, 60, 110)
HSV (OpenCV scale) = (H 107, S 209, V 110)
Greyscale = 54

Below each RGB colour we also show its HSV version (explained next) and its greyscale value.

Resizing

Big images are slow to process. Like the Stanford paper, we shrink every image to **256×256 pixels** (196,608 numbers) before doing anything else. We use OpenCV's "area" interpolation, which averages blocks of pixels so the small image still looks like the big one. Ships get even smaller, which is part of the difficulty.

Greyscale

A greyscale image has only one number per pixel: brightness, from 0 (black) to 255 (white). It's computed as a weighted mix of R, G and B (green counts most because our eyes are most sensitive to it):

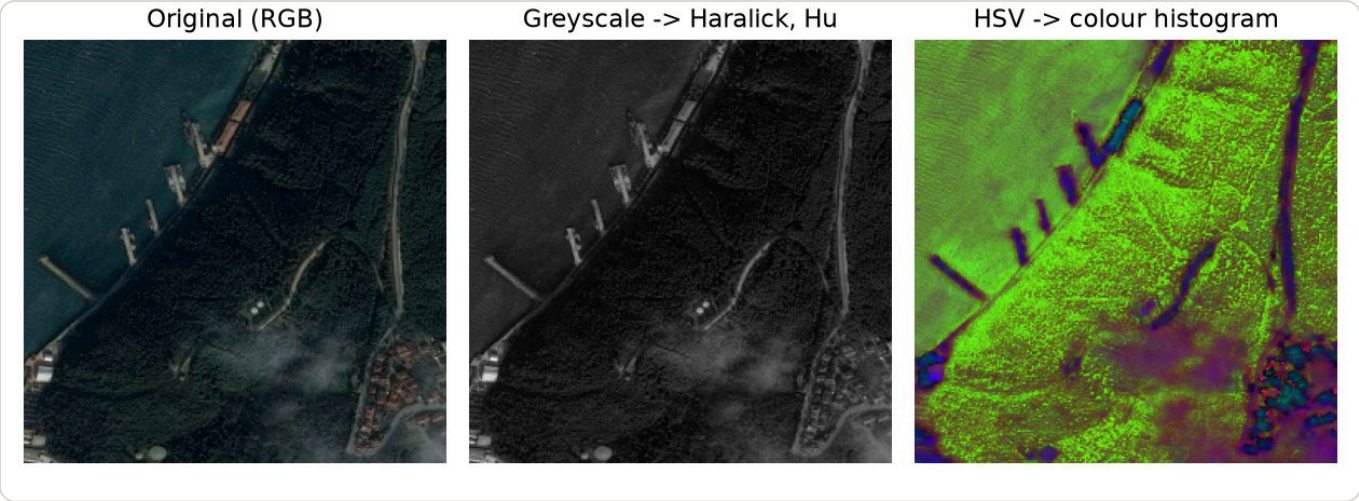
`grey = 0.299·R + 0.587·G + 0.114·B` . Two of our features (texture and shape) work on greyscale.

HSV colour space

RGB is how screens make colour, but it mixes up "what colour" with "how bright". If a cloud's shadow falls on the sea, all three of R, G and B drop, even though the sea is still "blue". **HSV** describes the same colour with three different numbers:

- **H**ue — which colour it is (red, yellow, green, blue…), as an angle around a colour wheel. OpenCV stores it as 0–179.
- **S**aturation — how vivid vs. greyish it is (0–255).
- **V**alue — how bright it is (0–255).

Because brightness lives in its own channel, HSV is less affected by lighting. That's why our colour feature uses HSV.



The same image as our features see it: original colour (left), greyscale used for texture and shape (middle), and HSV displayed as if it were RGB (right — the odd colours are expected; it's three different numbers per pixel shown in the RGB slots).

4 Our dataset

Where it comes from

The Kaggle Airbus Ship Detection Challenge provides:

- 192,555 labelled satellite images, each 768×768 RGB, about **31 GB** in total;
- a file called `train_ship_segmentations_v2.csv` describing where every ship is.

How the labels are stored: run-length encoding

The CSV has one row per ship. Each row gives the image name and the ship's pixels in a compressed format called **run-length encoding (RLE)**. If an image has no ships, it gets a single row with an empty mask.

RLE works like this. Number the pixels of the image 1, 2, 3, … going *down* the first column, then down the second column, and so on. Then write pairs of numbers: *start pixel, how many pixels in a row*.

WORKED EXAMPLE

In a tiny 5×5 image, the mask `7 3 12 3` means: pixels 7, 8, 9 (start 7, run of 3) and pixels 12, 13, 14 (start 12, run of 3) are ship. Pixels 7–9 are rows 2–4 of column 2; pixels 12–14 are rows 2–4 of column 3. So the ship is a 3×2 block:

1	6	11	16	21
2	**7**	**12**	17	22
3	**8**	**13**	18	23
4	**9**	**14**	19	24
5	10	15	20	25

RLE is just a space-saving trick: instead of listing every ship pixel, you list where each vertical run starts and how long it is.

From masks to yes/no labels

We don't need the outline of each ship — only whether there is one. So in `download_data.py` we count how many non-empty masks each image has:

```
labels = seg.groupby("ImageId").EncodedPixels.count()   # number of ship masks per image
has_ship = (n_ships > 0)                                 # 1 if at least one ship, else 0
```

In the full dataset only **22.1%** of images contain a ship. When one class is much more common than the other, the data is called **imbalanced**.

Why we used a subset — and which one

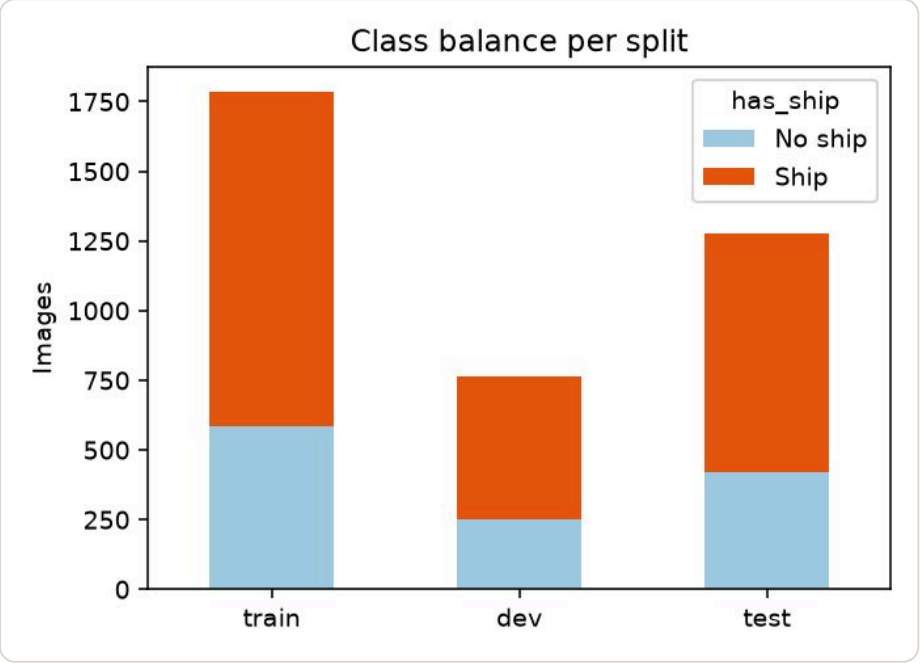
31 GB and 192k images is far too much for a laptop in one week. The Stanford students also used only 15,000 images. We had a **1 GB download limit**, so:

1. We downloaded the official label CSV (45 MB).
2. We started downloading randomly chosen images one by one. After 515 images Kaggle started refusing requests (a **rate limit** — a cap on how many downloads you may make in a short time).
3. We found a ready-made 494 MB collection of the same competition images on Kaggle (`aravjain007/aerial-ship-detection-dataset`) that downloads as one file: 3,325 images.
4. 11 images were in both, giving **3,829 images** in total.

Crucially, **every label comes from the official competition CSV**, not from the third-party collection, so the labels are trustworthy.

IN OUR PROJECT

Our 3,829 images are **67.2% ship and 32.8% no ship**. That's the opposite imbalance from the full data (22% ship), because the ready-made collection was assembled for ship detection and so contains mostly ship images. This matters for interpreting accuracy (Chapter 8) and for comparing with the paper (Chapter 14).



How many images of each class are in each part of our data.

VIVA TIP

Be upfront about the dataset: "We used 3,829 real competition images with official labels. Because of a 1 GB budget and Kaggle's rate limit, most came from a ready-made subset that is ship-enriched, so our class balance is 67% ships instead of 22%. We account for it by reporting the baseline, precision, recall and specificity, and by stratifying the split."

5 Train, dev and test sets

To check that a model generalises, we must test it on images it never saw while learning. We split our images into three groups:

Set	Images	Used for	Analogy
Training set	1,786	The model learns from these.	Textbook exercises with answers
Development (dev / validation) set	766	We use these to make decisions: which epoch's weights to keep, which decision threshold to use, when to lower the learning rate.	Mock exams you use to adjust how you study
Test set	1,277	Used exactly once, at the very end, to report the final scores.	The real final exam

The sizes follow the paper's **7 : 3 : 5** ratio (they used 7,000 / 3,000 / 5,000).

Why three sets and not two?

Every time we use some data to make a decision, the model's performance on that data becomes slightly optimistic — we have, in a sense, tuned to it. If we chose the threshold using the test set and then reported the test score, the score would be flattering. Keeping a separate dev set for decisions means the test set stays an honest, untouched measure.

Stratification

If we split randomly, one set might by bad luck get more ship images than another. A **stratified split** makes sure each set has the same proportion of each class — here about 67% ship in train, dev and test alike. We use scikit-learn's `train_test_split( … , stratify=has_ship)` twice: once to cut off the test set, once to split the rest into train and dev.

Data leakage

Leakage means information from the test data sneaks into training, making results look better than they really are. We avoid it by:

- never putting the same image in two sets;
- computing feature scaling statistics (Chapter 6) from training images only;
- choosing the threshold and best epoch on dev, never on test.

We also fix the random **seed** (to 229) so the split is identical every time the code runs, and we save the split in `data/labels.csv`.

6 Feature engineering (Route A)

Why not feed pixels straight into a classical model?

A 256×256×3 image is 196,608 numbers. Classical models struggle with that many inputs, and individual pixels are a terrible description of "is there a ship": move a ship by one pixel and almost every number changes, yet the answer hasn't changed.

So the traditional approach is **feature engineering**: a human designs a small set of numbers that summarise the image in a useful way, and the model learns from those. All of ours are **global descriptors** — numbers describing the whole image at once, not any particular spot.

We compute three kinds, exactly as the paper did.

1. Colour histogram ("His") — 512 numbers, describes colour

A **histogram** is a count of how many things fall into each bucket. A colour histogram counts how many pixels fall into each colour range.

1. Convert the image to HSV.
2. Divide each of H, S and V into 8 equal ranges. That gives 8 × 8 × 8 = **512 colour bins**.
3. For each pixel, find which bin its (H, S, V) falls into and add 1 to that bin.
4. Normalise the 512 counts (scale them so the vector has length 1), so the result doesn't depend on image size.

> **WORKED EXAMPLE**
> Imagine only 4 bins: *dark blue, light blue, white, brown*. An open-sea image of 100 pixels might give [90, 10, 0, 0]. A harbour image might give [40, 15, 20, 25]. A sea image with a white ship: [85, 9, 6, 0]. The model can learn that "some white on mostly dark blue" hints at a ship.

The histogram ignores *where* colours are — only how much of each there is. In our images an average of only about 65 of the 512 bins are non-zero; most bins are empty.

2. Haralick texture ("Ha") — 13 numbers, describes texture

Texture is the pattern of how brightness changes from pixel to pixel: smooth water, choppy waves, a busy port. In 1973 Robert Haralick proposed measuring it with a **grey-level co-occurrence matrix (GLCM)**.

A GLCM answers: *how often does a pixel of brightness i sit right next to a pixel of brightness j?*

WORKED EXAMPLE

A 4×4 greyscale image with only 4 brightness levels (0–3):

0	0	1	1
0	0	1	1
0	2	2	2
2	2	3	3

Look at every pair of horizontal neighbours (left pixel, right pixel). There are 12 pairs: (0,0) twice, (0,1) twice, (1,1) twice, (0,2) once, (2,2) three times, (2,3) once, (3,3) once.

Put the counts in a 4×4 table where row = left brightness, column = right brightness:

	j=0	**j=1**	**j=2**	**j=3**
i=0	2	2	1	0
i=1	0	2	0	0
i=2	0	0	3	1
i=3	0	0	0	1

Divide by 12 to turn counts into probabilities p(i, j). Now compute statistics. Two examples:

- **Contrast** = $\Sigma\, p(i,j)\cdot(i-j)^2$. Pairs with very different brightness add a lot. Here: $(2\cdot1 + 1\cdot4 + 1\cdot1)/12$ = **0.58**. Smooth sea → low contrast.
- **Homogeneity** = $\Sigma\, p(i,j) / (1+(i-j)^2)$. Pairs with equal brightness add the most. Here: $(8 + 2/2 + 1/5 + 1/2)/12$ = **0.81**. Smooth sea → high homogeneity.

Haralick defined 13 such statistics (contrast, correlation, entropy, homogeneity, energy and others). We compute the GLCM in four directions (→, ↓, ↘, ↙) with 256 grey levels using the `mahotas` library, compute the 13 statistics for each direction, and average the four results. Result: 13 numbers per image.

IN OUR PROJECT

Averaged over our training images, ship images have clearly higher *correlation* (0.76 vs 0.52) and *variance* (654 vs 389) than no-ship images. In words: ship images tend to contain more structure. Part of that is the ships, but part is probably that ship images more often show coasts and harbours. A model can partly learn "coast vs. open sea" instead of "ship vs. no ship" — a known risk called **shortcut learning**.

3. Hu moments ("Hu") — 7 numbers, describes shape

An **image moment** is a weighted sum of pixel brightnesses. The simplest ones tell you basic facts:

- M_{00} = sum of all brightness values → the total "mass" of brightness;
- M_{10}/M_{00} and M_{01}/M_{00} → the centre of brightness (like the centre of gravity);

- higher-order moments → how spread out and stretched the brightness is, and in which direction.

In 1962 Ming-Kuei Hu combined moments into **7 numbers that don't change if the image is shifted, scaled or rotated**. They're great for describing the shape of a single object (like a letter on a page). For a whole satellite scene full of sea, coast and cloud, they're a weak description — and our results confirm it: Hu moments were the least useful feature for every model.

4. Raw pixels — the "no feature engineering" baseline

To measure whether feature engineering helps, we also try the most naive input: shrink the image to 32×32×3 and use its 3,072 pixel values directly (scaled to 0–1). The paper used the full 256×256 image; we shrank it further so k-NN and SVM stay fast on a laptop.

Feature combinations

We test each model with 8 inputs: each descriptor alone (His, Ha, Hu), each pair (His+Ha, Ha+Hu, Hu+His), all three together (512+13+7 = **532 numbers**) and raw pixels. Combining just means putting the vectors side by side.

Standardisation

Our features have wildly different scales: histogram bins are around 0–1, Haralick variance is in the hundreds, Hu moments can be 0.000000001. Models that measure distances (k-NN, SVM) would be dominated by the biggest numbers. So we **standardise** each feature: subtract its mean and divide by its standard deviation.

WORKED EXAMPLE

A feature has training values 2, 4, 6. Mean = 4, standard deviation ≈ 1.63. Standardised: (2−4)/1.63 = −1.22, (4−4)/1.63 = 0, (6−4)/1.63 = +1.22. Now every feature has mean 0 and spread 1.

COMMON MISTAKE

Computing the mean and standard deviation on *all* the data, including test. That leaks test information into training. We use scikit-learn's `StandardScaler` inside a `Pipeline`, so it learns its mean and standard deviation from the training data only — and, during cross-validation, from each fold's training part only.

7 The five classical models

Each of these takes the feature vector of an image and outputs "ship" or "no ship". They are called *classical* or *traditional* to distinguish them from neural networks. We use the implementations in **scikit-learn**, the standard Python ML library.

To build intuition, imagine each image as a **point in space**: with 2 features, a point on a flat sheet; with 532 features, a point in a 532-dimensional space (impossible to picture, but the maths is the same). Ship images and no-ship images form two clouds of points. Every classifier is a different way of drawing a **decision boundary** between the clouds.

7.1 k-Nearest Neighbours (k-NN)

> **IN PLAIN WORDS**
> "Tell me who your neighbours are and I'll tell you who you are." To classify a new image, find the 5 training images whose features are most similar, and go with the majority.

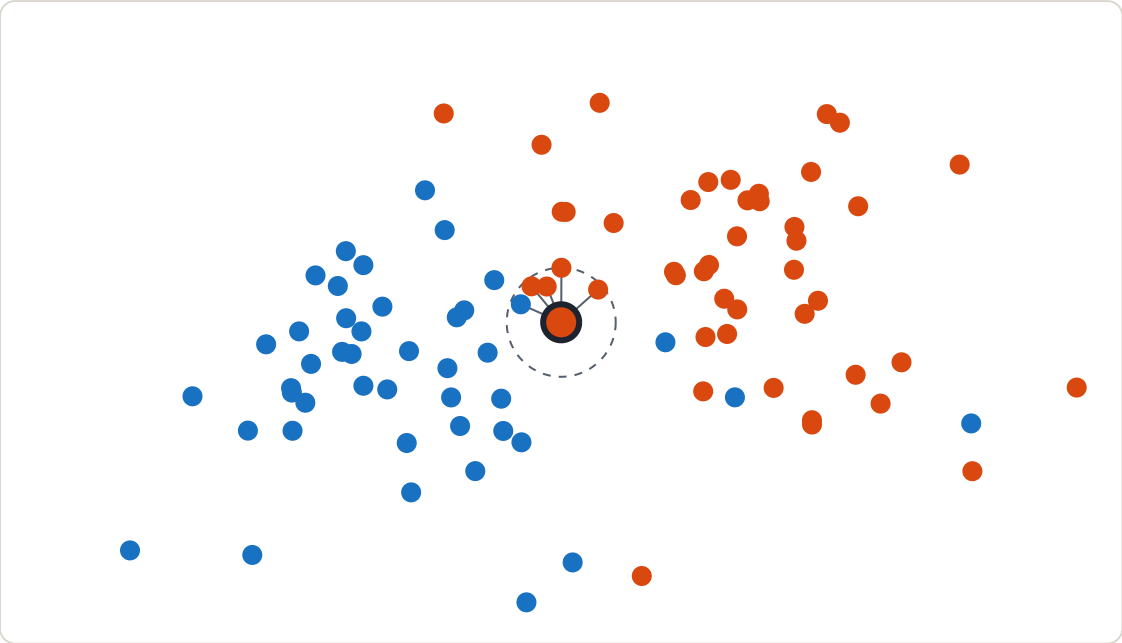
"Most similar" means smallest **Euclidean distance** — ordinary straight-line distance, generalised to many dimensions: $d(a,b) = \sqrt{(a_1-b_1)^2 + (a_2-b_2)^2 + \dots + (a_n-b_n)^2}$

Interactive demo — shown here in its starting state. Open Beginners_Guide.html in a browser to use it.

Try it: k-NN on a toy 2-D dataset

Each dot is a made-up "image" described by two features. ● = ship, ● = no ship. Click anywhere to classify a new point by its k nearest neighbours.

k = 5 Neighbours: **4** ship, **1** no ship → predicted **SHIP**



Training is trivial: just store all training examples. **Predicting** is slow, because each new image must be compared with every training image. k-NN also suffers from the **curse of dimensionality**: in hundreds of dimensions, all points end up roughly equally far apart, so "nearest" means less.

7.2 Gaussian Naive Bayes (NB)

Naive Bayes is built on **Bayes' theorem**, a rule for updating a belief when you see evidence:

```
P(ship | features) = P(features | ship) × P(ship) / P(features)
```

- **P(ship)** — the *prior*: how common ships are before looking (67% in our training data).
- **P(features | ship)** — the *likelihood*: how typical these feature values are among ship images.
- **P(ship | features)** — the *posterior*: what we actually want.

The "naive" part: it assumes every feature is **independent** of the others once you know the class, so P(features | ship) is just the product of each feature's individual probability. "Gaussian" means it models each feature, within each class, as a bell curve (a normal distribution) with its own mean and spread.

WORKED EXAMPLE

Suppose 60% of images have ships. Among ship images, 50% have "lots of white"; among no-ship images, 10% do. An image has lots of white. Then:
ship score = 0.5 × 0.6 = 0.30; no-ship score = 0.1 × 0.4 = 0.04.
P(ship | lots of white) = 0.30 / (0.30 + 0.04) ≈ **88%**.

IN OUR PROJECT

Naive Bayes was by far our worst model whenever the colour histogram was included (52.6% with His alone — worse than always answering "ship"!), but decent on Haralick alone (73.4%). Why? Its two assumptions are badly broken by histogram bins: neighbouring bins are strongly *correlated* (not independent), and most bins are almost always zero (not bell-shaped). Multiplying 512 badly-estimated probabilities produces wildly over-confident wrong answers. The Stanford paper saw exactly the same pattern (0.49 with His, 0.84 with Ha).

7.3 Linear Discriminant Analysis (LDA)

IN PLAIN WORDS

Find the single direction in feature space along which the two classes are most separated, project every image onto that line, and put a cut-off point on it.

LDA assumes each class is a Gaussian "cloud" and that both clouds have the same shape (the same **covariance matrix** — a table describing how features vary together). Under those assumptions, the best boundary is a straight line (a flat **hyperplane** in many dimensions). It needs no tuning and is fast, but cannot draw curved boundaries.

In our results LDA did very well with all features (85.4%) but badly on raw pixels (53.7%).

7.4 Decision trees and the Random Forest

A **decision tree** is a flowchart of yes/no questions about the features:

```
Is histogram bin #37 > 0.12 ?
├── yes → Is Haralick correlation > 0.64 ?
│         ├── yes → SHIP
│         └── no  → NO SHIP
└── no  → ...
```

During training the tree chooses, at each step, the question that best separates ships from non-ships (measured by **Gini impurity** — how mixed the classes are in each resulting group). A single tree tends to overfit: it keeps asking questions until it has memorised the training data.

A **Random Forest** fixes this by growing many different trees and letting them **vote**:

1. **Bootstrap sampling:** each tree is trained on a random sample of the training images drawn *with replacement* (some images appear twice, some not at all).
2. **Random feature subsets:** at each question, a tree may only choose among a random subset of features (by default $\sqrt{532} \approx 23$ of them).
3. **Vote:** to classify an image, all trees answer and the majority wins.

Because the trees are trained differently, they make *different* mistakes, and the errors cancel out in the vote. This is called an **ensemble**, and the bootstrap-then-average idea is called **bagging** (bootstrap aggregating).

IN OUR PROJECT

We use 70 trees (the paper's poster setting). Random Forest was our best classical model: **91.9%** test accuracy with all features, and **92.4% ± 1.7%** in 10-fold cross-validation — the highest average and one of the smallest spreads. We can also ask the forest how much it relied on each feature group: **82%** of its decisions came from colour-histogram bins, **15%** from Haralick texture, and only **3%** from Hu moments.

VIVA TIP

"Why did Random Forest win?" — It can draw non-linear boundaries; averaging many de-correlated trees reduces variance; it doesn't care about feature scale; and it naturally ignores useless features, which matters when most of the 512 histogram bins are empty.

7.5 Support Vector Machine (SVM)

IN PLAIN WORDS

Of all the lines that separate the two classes, pick the one that leaves the widest possible gap ("margin") on both sides. A wide gap means new points near the boundary are less likely to be misclassified.

The training points closest to the boundary — the ones that "support" it — are called **support vectors**; only they determine where the boundary goes. Real data is rarely perfectly separable, so a **soft-margin** SVM allows some points on the wrong side, with a penalty controlled by a setting called `C` (we use the default, 1.0).

A straight line often isn't enough. The **kernel trick** lets the SVM behave as if the data had been mapped into a much higher-dimensional space where a straight boundary works — without ever computing that mapping. We use the **RBF (radial basis function) kernel**, which measures similarity as

$K(a,b) = \exp(-\gamma \cdot \|a-b\|^2)$ — close points are similar (≈ 1), far points are not (≈ 0). The resulting boundary in the original space can be curved.

Our SVM scored 85.0% with all features.

8 Measuring how good a model is

The confusion matrix

Treat "ship" as the **positive** class. Every prediction falls into one of four boxes:

	Model says "No ship"	**Model says "Ship"**
Really no ship	**True Negative (TN)** — correctly said empty	**False Positive (FP)** — false alarm
Really a ship	**False Negative (FN)** — missed ship	**True Positive (TP)** — correctly found

This 2×2 table is the **confusion matrix**. Every other metric is calculated from it.

Accuracy — and why it can lie

```
accuracy = (TP + TN) / total
```

The fraction of all predictions that are correct. Simple, but misleading on imbalanced data. In our test set 67.2% of images have ships, so a "model" that always answers *ship* scores **67.2% accuracy** while being completely useless. This is the **majority-class baseline**: any real model must beat it.

Precision, recall and specificity

Metric	**Formula**	**Question it answers**
Precision	TP / (TP + FP)	When the model says "ship", how often is it right?
Recall (sensitivity)	TP / (TP + FN)	Of all real ship images, how many did it find?
Specificity	TN / (TN + FP)	Of all empty images, how many did it correctly call empty?
F1 score	2·P·R / (P + R)	A single number balancing precision and recall.

> **WORKED EXAMPLE — OUR TL-CNN ON 1,277 TEST IMAGES**
> TN = 387, FP = 32, FN = 28, TP = 830.
> Accuracy = (830+387)/1277 = **95.3%**. Precision = 830/(830+32) = **96.3%**. Recall = 830/(830+28) = **96.7%**. Specificity = 387/(387+32) = **92.4%**.

Probabilities and the decision threshold

Most of our models don't just say yes or no; they output a **probability** P(ship) between 0 and 1. We turn that into a decision with a **threshold**: say "ship" if P(ship) ≥ threshold. The default is 0.5, but nothing says 0.5 is best:

- **Lower threshold** → more images called "ship" → recall goes up, but more false alarms (precision and specificity go down).
- **Higher threshold** → fewer "ship" calls → fewer false alarms, but more missed ships.

Following the paper, we try thresholds 0.1, 0.2, ..., 0.9 on the **dev** set, keep the one with the best dev accuracy, and only then apply it to the test set. That gave 0.5 for TL-CNN and 0.6 for SIM-CNN.

Interactive demo — shown here in its starting state. Open Beginners_Guide.html in a browser to use it.

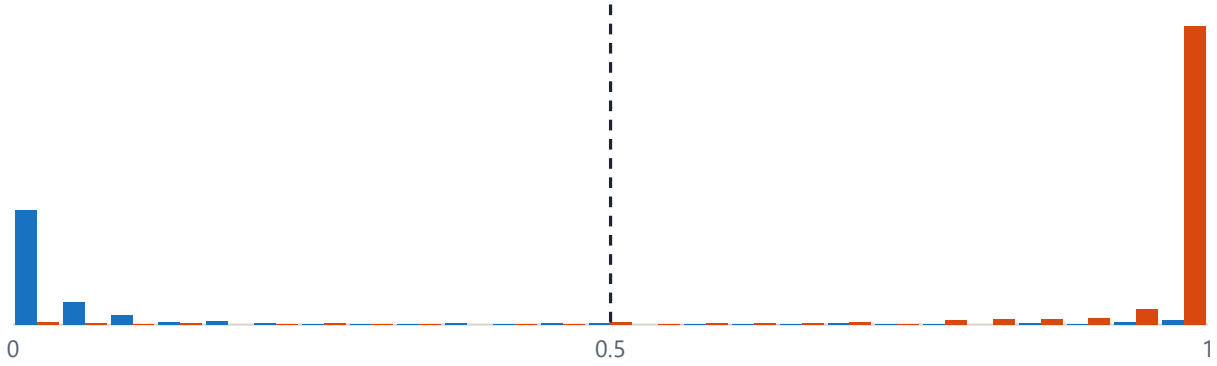
Try it: move the threshold on our real test predictions

These are the actual probabilities our trained models gave to the 1,277 test images.

Model: TL-CNN (transfer learning) | Threshold: 0.50

	Says no ship	Says ship
Really no ship	387 TN	32 FP · false alarm
Really ship	28 FN · missed	830 TP

95.3% Accuracy
96.3% Precision
96.7% Recall
92.4% Specificity
67.2% Always-"ship" baseline



Histogram of P(ship): ■ real ship images, ■ real empty images. Everything right of the dashed line is called "ship". The starting threshold for each model is the one we actually used. A good model pushes the two colours apart.

ROC curve and AUC

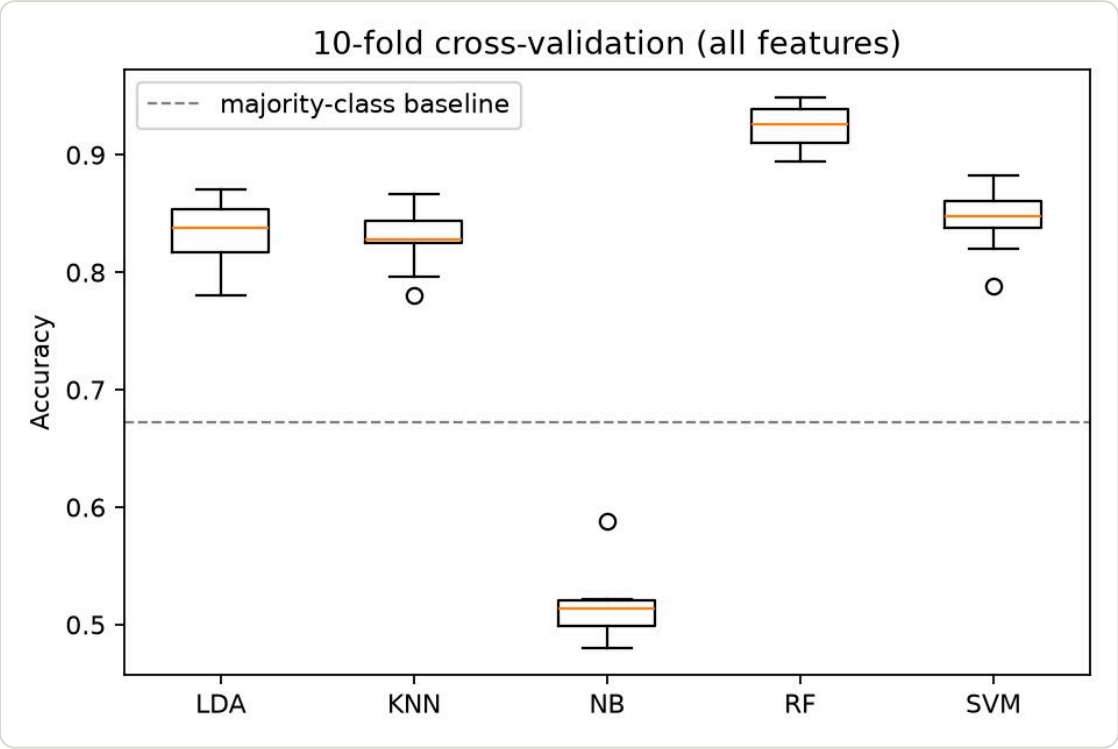
If you slide the threshold from 1 down to 0 and plot recall (true-positive rate) against 1 – specificity (false-positive rate) at each step, you get the **ROC curve**. The **area under it (AUC)** summarises the model across all thresholds. It has a neat meaning: *the probability that a randomly chosen ship image gets a higher P(ship) than a randomly chosen empty image*. 0.5 = random guessing, 1.0 = perfect. TL-CNN's AUC was 0.982; SIM-CNN's 0.928.

Cross-validation

A single train/test split gives one number, which depends partly on luck — which images happened to land in test. **k-fold cross-validation** gives a more reliable picture:

1. Split the data into k equal parts ("folds"); we use k = 10.
2. Train on 9 folds, test on the remaining one.
3. Repeat 10 times so each fold is the test fold once.
4. Report the average and the spread (standard deviation) of the 10 scores.

We run it on train + dev together (2,552 images), never touching the test set, using *stratified* folds. A model with a high average and a small spread is both good and reliable.



Our 10-fold cross-validation results. Each box shows the spread of the 10 scores; the line inside is the median. Random Forest is highest and tight; Naive Bayes is far below the dashed majority-class baseline.

9 Neural networks from zero

Route B skips hand-made features entirely: a **neural network** looks at the raw pixels and learns its own features. To understand our two networks we need to build up from the smallest piece.

9.1 One artificial neuron

A **neuron** takes several input numbers, multiplies each by a **weight**, adds them up, adds a **bias**, and passes the result through an **activation function**:

$$\text{output} = \text{activation}(w_1 \cdot x_1 + w_2 \cdot x_2 + \dots + w_n \cdot x_n + b)$$

The weights say how much each input matters (and whether it pushes the output up or down); the bias shifts the result. These weights and biases are the network's **parameters** — the numbers training adjusts.

> **WORKED EXAMPLE**
> Inputs x = (0.8, 0.2), weights w = (1.5, −2.0), bias b = 0.1.
> Sum = 1.5·0.8 + (−2.0)·0.2 + 0.1 = 1.2 − 0.4 + 0.1 = 0.9.
> With the ReLU activation (below), output = max(0, 0.9) = 0.9.

9.2 Activation functions

Without an activation function, stacking neurons would only ever produce a straight-line (linear) function, no matter how many layers — not enough to recognise ships. Activations add the **non-linearity** that lets networks learn complex patterns.

- **ReLU** (Rectified Linear Unit): $\text{ReLU}(z) = \max(0, z)$. Negative → 0, positive → unchanged. Simple, fast, and the standard choice for hidden layers. We use it everywhere inside our networks.
- **Sigmoid**: $\sigma(z) = 1 / (1 + e^{-z})$. Squashes any number into the range 0 to 1, so it can be read as a probability. We use it once, at the very end, to turn the network's final number into P(ship).

9.3 Layers and networks

Put many neurons side by side and you have a **layer**. Feed one layer's outputs into the next and you have a **network**. A **fully connected** (or *dense*, or *linear*) layer connects every input to every neuron. The layers between input and output are **hidden layers**. "Deep learning" just means networks with many layers.

9.4 The loss function: measuring wrongness

To improve, the network needs a single number saying how wrong it currently is: the **loss**. For yes/no problems with a probability output, the standard is **binary cross-entropy (BCE)**:

```
loss = -[ y·ln(p) + (1−y)·ln(1−p) ]
```

where y is the true label (1 = ship, 0 = no ship) and p is the predicted P(ship). If the image has a ship (y = 1), the loss is −ln(p): small when p is near 1, huge when p is near 0. It punishes *confident wrong answers* very heavily.

Interactive demo — shown here in its starting state. Open Beginners_Guide.html in a browser to use it.

Try it: binary cross-entropy

True label: 1 — ship

Predicted P(ship): 0.90

0.105
loss

The training loss we report is the average BCE over all images.

9.5 Gradient descent: how learning actually happens

> **IN PLAIN WORDS**
> You're standing on a foggy hillside and want to reach the valley. You can't see far, but you can feel which way the ground slopes under your feet. So you take a small step downhill, feel again, take another step… Eventually you reach the bottom.

The "hillside" is the loss, plotted against the network's parameters. The **gradient** is the slope: for each parameter, how much the loss would change if you nudged that parameter. **Gradient descent** repeatedly moves every parameter a little bit in the downhill direction:

```
new weight = old weight − learning_rate × gradient
```

The **learning rate** sets the step size. Too small and learning crawls; too large and you overshoot the valley and bounce around or diverge.



9.6 Backpropagation

A network has millions of parameters; how do we get the gradient for each one? **Backpropagation** uses the **chain rule** from calculus: start at the loss, work backwards through the network layer by layer, and at each layer work out how much each parameter contributed to the error. It's just an efficient bookkeeping method for computing all the gradients at once. PyTorch does it automatically when we call `loss.backward()`.

9.7 Batches, epochs and Adam

- Computing the gradient on all 1,786 training images before every step would be slow. Instead we use **mini-batches** of 64 images: compute the loss on 64 images, take one step, move on to the next 64. This is **stochastic gradient descent**.
- One full pass through all training images is an **epoch** (≈28 steps for us). We train for **30 epochs**, like the paper.
- **Adam** is a smarter version of gradient descent that we use as our **optimiser**. It keeps a running average of recent gradients (*momentum* — like a ball rolling downhill that doesn't stop at every bump) and gives each parameter its own adaptive step size. Our starting learning rate is 0.001.
- **Learning-rate decay:** when the dev loss hasn't improved for 2 epochs, we halve the learning rate (PyTorch's `ReduceLROnPlateau`). Big steps early, small careful steps later. For SIM-CNN the rate ended at 0.00025; for TL-CNN it shrank all the way to about 0.000004.

IN OUR PROJECT — THE TRAINING LOOP IN CODE

```
for each epoch (30 times):
    for each batch of 64 training images:
        logits = model(images)               # forward pass: predictions
        loss = BCEWithLogitsLoss(logits, y)  # how wrong?
        opt.zero_grad(); loss.backward()     # backpropagation: gradients
        opt.step()                           # Adam updates every weight
    measure loss on the dev set
    if dev loss is the best so far: save the weights
```

`BCEWithLogitsLoss` is sigmoid + binary cross-entropy combined in one numerically safer step; the network's raw final number (before sigmoid) is called a **logit**.

10 Convolutional neural networks (CNNs)

Why not a plain fully connected network?

A fully connected layer from a 256×256×3 image to just 128 neurons would need 196,608 × 128 ≈ **25 million weights** in the first layer alone. Worse, it would treat a ship in the top-left corner and the same ship in the bottom-right as completely unrelated patterns, and would have to learn each separately. Images need something smarter.

10.1 Convolution: a sliding pattern detector

A **convolution** slides a small grid of weights — a **filter** (or *kernel*), 3×3 in our case — across the image. At each position it multiplies the 9 filter weights by the 9 pixels underneath, adds them up (plus a bias), and writes the result into an output grid called a **feature map**. A filter "lights up" wherever the image contains the pattern it's tuned to.

- **Local connectivity:** each output only looks at a small neighbourhood — ships are local objects.
- **Weight sharing:** the same 9 weights are used at every position, so a ship detector works anywhere in the image, and the layer needs very few parameters.
- The filter weights are *not* designed by hand — they're learned by gradient descent, just like any other weight.

Interactive demo — shown here in its starting state. Open Beginners_Guide.html in a browser to use it.

Try it: a 3×3 filter sliding over a tiny "satellite image"

The left grid is a 10×10 greyscale patch: dark sea with a small bright "ship". Pick a filter and see the feature map it produces (with padding, so the output stays 10×10).

Filter [Bright-spot detector] ☑ apply ReLU ☐ then 2×2 max-pool

Input

30	34	38	33	37	32	36	31	35	30
37	32	36	31	35	30	34	38	33	37
35	30	34	38	33	37	32	36	31	35
33	37	32	36	120	35	30	34	38	33
31	35	30	220	220	220	220	32	36	31
38	33	37	220	220	220	160	30	34	38
36	31	35	30	34	38	33	37	32	36
34	38	33	37	32	36	31	35	30	34
32	36	31	35	30	34	38	33	37	32
30	34	38	33	37	32	36	31	35	30

Filter weights

-1	-1	-1
-1	8	-1
-1	-1	-1

Feature map (10×10)

137	99	138	87	135	84	123	72	111	135
135	0	18	0	9	0	0	36	0	132
111	0	0	0	0	0	0	18	0	108
96	36	0	0	121	0	0	0	36	93
72	9	0	845	469	535	999	0	18	69
138	0	0	934	558	615	450	0	0	135
114	0	0	0	0	0	0	0	0	120
99	36	0	36	0	18	0	9	0	105
84	18	0	9	0	0	36	0	36	90
138	105	135	93	132	81	120	69	117	136

Notice the bright-spot detector lights up strongly on the ship. It also gives fairly large values along the border: there the filter overlaps the zero padding, so the "surroundings" look dark and the edge pixel looks like a bright spot. Real networks learn to cope with this.

10.2 Padding and channels

At the image border the filter would hang off the edge. **Padding** adds a 1-pixel frame of zeros around the image so the output keeps the same width and height (we use `padding=1`).

A real convolution layer has *many* filters, each producing its own feature map. Our first layer has 16 filters, so a 256×256×3 image becomes a 256×256×16 stack. Each filter looks at *all* input channels at once, so a first-layer filter is really 3×3×3 = 27 weights plus 1 bias.

10.3 Max-pooling

Max-pooling shrinks a feature map by keeping only the largest value in each 2×2 block. Width and height halve. This reduces computation and makes the network tolerant to small shifts — if a ship moves by one pixel, the max in its block often stays the same.

> **WORKED EXAMPLE**
> 4×4 → 2×2: the blocks [1, 3; 2, 0], [5, 1; 0, 2], [0, 0; 1, 4], [6, 2; 3, 1] become [3, 5; 4, 6].

10.4 Stacking layers: from edges to ships

Early convolution layers learn simple patterns (edges, bright spots, colour blobs). Later layers combine those into more complex ones (a bright elongated shape with a wake). Because each pooling halves the image, deeper filters "see" larger areas of the original image. At the end, the feature maps are **flattened** into one long vector and fed into fully connected layers that make the final decision.

10.5 Our SIM-CNN, layer by layer

Layer	Output shape	Parameters	How the count is computed
Input image	256 × 256 × 3	0	
Conv 3×3, 16 filters + ReLU + max-pool	128 × 128 × 16	448	(3·3·3 + 1) × 16
Conv 3×3, 32 filters + ReLU + max-pool	64 × 64 × 32	4,640	(3·3·16 + 1) × 32
Conv 3×3, 64 filters + ReLU + max-pool	32 × 32 × 64	18,496	(3·3·32 + 1) × 64
Conv 3×3, 128 filters + ReLU + max-pool	16 × 16 × 128	73,856	(3·3·64 + 1) × 128
Flatten	32,768	0	16·16·128
Fully connected + ReLU + dropout 0.5	128	4,194,432	32,768 × 128 + 128
Fully connected → sigmoid	1	129	128 + 1
Total		**4,292,001**	**exactly the paper's number**

VIVA TIP

Memorise the conv formula: **(kernel height × kernel width × input channels + 1 bias) × number of filters**. And notice that **98% of SIM-CNN's parameters are in the first fully connected layer** — the convolutions themselves are tiny. That's a sign of why it needs lots of data.

11 Fighting overfitting

SIM-CNN has 4.3 million parameters and we only have 1,786 training images. With that many knobs, the network could easily memorise every training image. We use four defences:

Data augmentation

A ship is still a ship if you flip, rotate or slightly zoom the photo. So every time an image is shown to SIM-CNN during training we apply random changes: horizontal and vertical **flips**, **rotation** up to ±20°, **shifts** up to ±10%, **zoom** ±10%, a little **shear** (slanting), and **brightness** ±20%. The network effectively sees a slightly different image each epoch, which makes memorising much harder. We only augment training images — dev and test images are left untouched.

Dropout

During training, **dropout** randomly switches off a fraction of neurons (half, in our case) on every step. The network can't rely on any single neuron, so it learns more robust, spread-out patterns. At prediction time all neurons are used.

Batch normalisation

Batch normalisation rescales a layer's inputs to have zero mean and unit spread within each batch, then lets the network learn the best scale and shift. It stabilises and speeds up training. We use it in the TL-CNN head.

Keeping the best checkpoint (early stopping)

After every epoch we measure the loss on the dev set and save the weights only if it's the best so far. If the network starts overfitting later, we still keep the earlier, better weights. This is a form of **early stopping**.

> **IN OUR PROJECT**
> The TL-CNN head's dev loss was lowest at **epoch 5** and then slowly rose while its training accuracy kept climbing towards 99% — textbook overfitting. Because of checkpointing, the weights we use are from epoch 5. SIM-CNN's dev loss kept improving almost to the end; its best epoch was **29**.

12 Transfer learning (our TL-CNN)

> **IN PLAIN WORDS**
> Someone who already knows how to drive a car learns to drive a truck much faster than someone who has never driven. Transfer learning reuses knowledge a network learned on one big task for a new task.

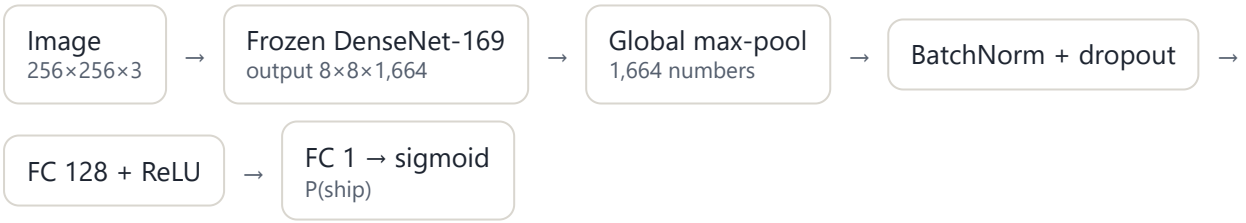
ImageNet and DenseNet-169

ImageNet is a famous dataset of about 1.2 million everyday photos in 1,000 categories (dogs, cars, cups…). Networks trained on it learn very general visual building blocks in their early and middle layers: edges, textures, corners, simple shapes.

DenseNet-169 is one such network, 169 layers deep. In a DenseNet, each layer receives the outputs of *all* earlier layers in its block ("dense connections"), which helps information and gradients flow through a very deep network. Its last convolutional stage outputs 1,664 feature maps. PyTorch's `torchvision` library provides it already trained on ImageNet.

Freezing the backbone

We use DenseNet-169 as a fixed **feature extractor** (the "backbone"): its roughly 12.5 million weights are **frozen** — never changed. On top we add a small new classifier (the "head"), and only the head is trained:



Global max-pooling takes each of the 1,664 feature maps (8×8) and keeps only its largest value, giving 1,664 numbers: "how strongly did each pattern appear anywhere in the image?"

Trainable parameters: BatchNorm 2 × 1,664 = 3,328; first FC layer 1,664 × 128 + 128 = 213,120; last layer 129. **Total 216,577** — again exactly the paper's number.

Before images go into DenseNet we normalise them with ImageNet's average colour and spread, because that's what the network saw during its original training.

Our speed trick: caching

Because the backbone is frozen, its output for a given image never changes. So we run every image through DenseNet **once**, save the 1,664 numbers, and train the head on those saved numbers. Each TL-CNN epoch then takes under a second instead of minutes. The trade-off: we can't apply data augmentation to TL-CNN (an augmented image would need a fresh pass through DenseNet).

VIVA TIP

"Why freeze instead of fine-tune?" — Freezing means only 216,577 weights are trained, which is fast on a CPU and very hard to overfit with 1,786 images. **Fine-tuning** (also updating some DenseNet layers) usually does better but needs more compute and more care. It's in our future work.

13 Our results, explained

All numbers are on the same **1,277 test images**, which no model saw during training or tuning.

13.1 Classical models: which features help?

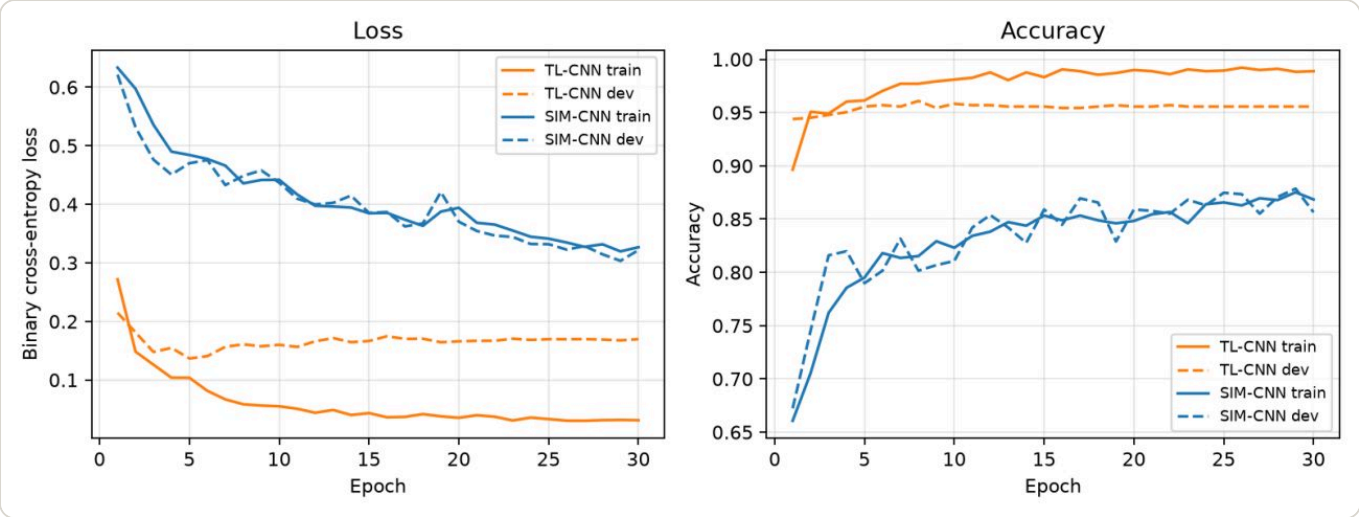
Test accuracy for each model (rows) and input (columns):

	His	Ha	Hu	His+Ha	Ha+Hu	Hu+His	All three	Raw pixels
LDA	0.748	0.784	0.683	0.849	0.785	0.756	0.854	0.537
k-NN	0.785	0.829	0.674	0.838	0.825	0.795	0.839	0.735
NB	0.526	0.734	0.672	0.525	0.676	0.531	0.532	0.625
RF	0.915	0.854	0.733	0.918	0.854	0.913	**0.919**	0.821
SVM	0.761	0.832	0.681	0.848	0.825	0.762	0.850	0.808

How to read it:

- **Feature engineering helps most models.** Going from raw pixels to all three descriptors: LDA +31.6 points, k-NN +10.4, RF +9.8, SVM +4.2. The hand-made numbers make the "is there a ship" signal much easier to find than raw pixels do.
- **...but not Naive Bayes** (−9.3), because of the broken independence assumption explained in Chapter 7.
- **"More is less" sometimes.** Adding features doesn't always help: NB is best with Haralick alone, and for k-NN and SVM adding Hu moments to His+Ha changes almost nothing (0.838 → 0.839, 0.848 → 0.850).
- **Colour is the strongest single descriptor** for Random Forest (0.915 alone); **Hu moments are the weakest** for every model (≈0.67–0.73, barely above the 0.672 baseline).

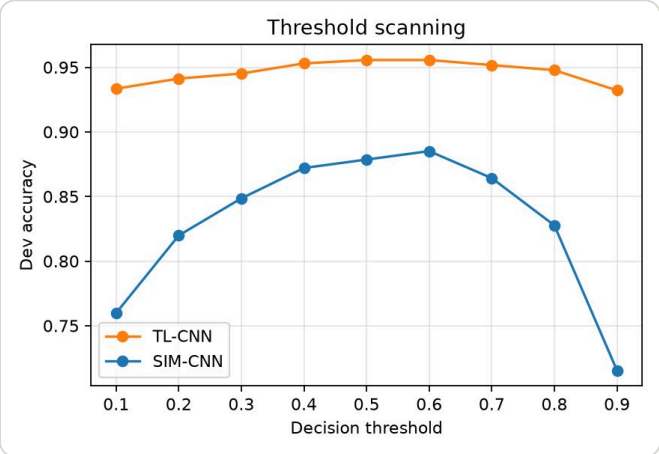
13.2 The CNNs during training



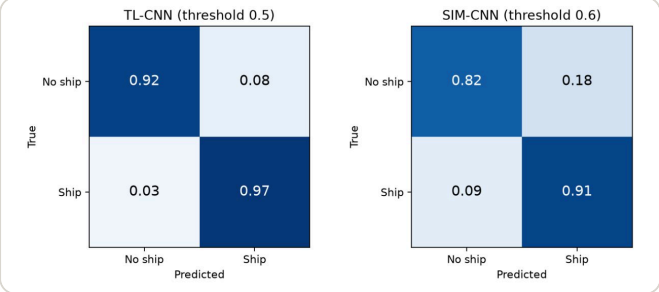
Loss (left) and accuracy (right) per epoch. Solid = training images, dashed = dev images. Orange = TL-CNN, blue = SIM-CNN.

- **TL-CNN** learns very fast (it starts with good features). Its training loss keeps falling, but its dev loss bottoms out around epoch 5 and then creeps up: overfitting. Checkpointing saves the epoch-5 weights.
- **SIM-CNN** learns slowly and steadily from random weights. Training and dev curves stay close together (augmentation and dropout are doing their job), and the dev loss was still falling at epoch 30 — it hadn't finished learning. More epochs might have helped; we stopped at 30 to match the paper.

13.3 Thresholds and confusion matrices



Dev accuracy at thresholds 0.1–0.9. TL-CNN is flat and high; SIM-CNN peaks at 0.6.

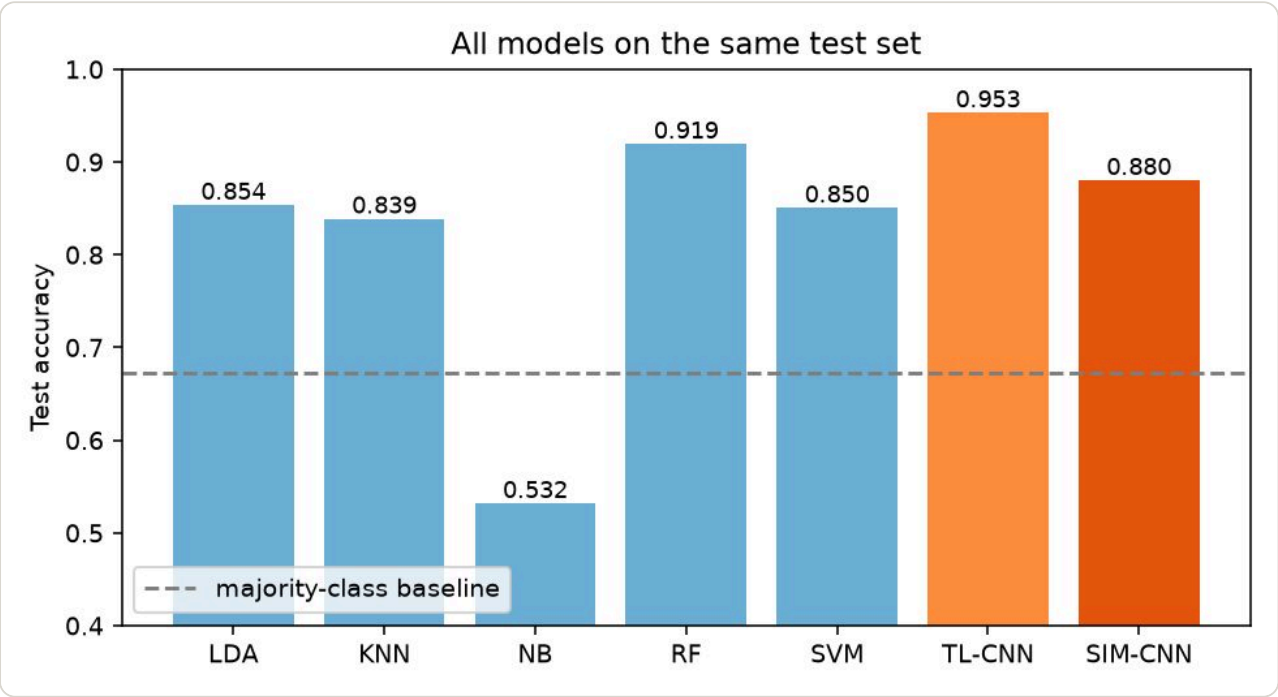


Normalised confusion matrices on test: each row sums to 1, so the diagonal shows specificity (top-left) and recall (bottom-right).

SIM-CNN at the default 0.5 would have scored 87.1%; the dev-chosen 0.6 gave 88.0% on test, mostly by cutting false alarms (specificity 75.9% → 81.9%).

13.4 Everything side by side

Model	Accuracy	Precision	Recall	Specificity	Notes
Always say "ship"	67.2%	67.2%	100%	0%	baseline to beat
LDA (all features)	85.4%	87.0%	92.0%	—	CV 83.4 ± 2.5%
k-NN (all features)	83.9%	85.6%	91.4%	—	CV 82.9 ± 2.4%
Naive Bayes (all features)	53.2%	91.4%	33.4%	—	CV 51.5 ± 2.8%; below baseline
Random Forest (all features)	**91.9%**	93.2%	94.9%	—	CV 92.4 ± 1.7%; best classical
SVM (all features)	85.0%	84.3%	95.5%	—	CV 84.6 ± 2.5%
TL-CNN (threshold 0.5)	**95.3%**	96.3%	96.7%	92.4%	AUC 0.982; best overall
SIM-CNN (threshold 0.6)	88.0%	91.1%	91.0%	81.9%	AUC 0.928



Test accuracy of every model; the dashed line is the 67.2% majority-class baseline.

13.5 Why did things turn out this way?

- **TL-CNN won** because it didn't have to learn vision from scratch: DenseNet's features, learned from 1.2 million images, already recognise edges, textures and blobs. With only 1,786 training images, borrowing that knowledge is a big advantage.
- **SIM-CNN came below Random Forest.** A 4.3-million-parameter network trained from random weights needs far more examples than we had, and it hadn't fully converged after 30 epochs. Most likely, more data and more epochs would close the gap. (This is our best explanation — we didn't run an experiment varying the training-set size to prove it.)
- **Random Forest did remarkably well** for a model that sees only 532 summary numbers, mostly colour. Part of that may be shortcut learning (coast vs. open sea) rather than truly spotting ships.

- **Naive Bayes failed** because its assumptions don't fit histogram data.

> **VIVA TIP — "WHICH MODEL WOULD YOU DEPLOY?"**
> On accuracy, TL-CNN (95.3%). But Random Forest (91.9%) is far cheaper to train and run and needs no deep-learning setup, so it's a sensible choice if compute is limited. The real answer depends on the cost of a missed ship versus a false alarm — which would also change the threshold.

14 Us vs. the Stanford paper

	Stanford paper (2018)	Our project
Images (train / dev / test)	7,000 / 3,000 / 5,000	1,786 / 766 / 1,277
Class balance	about 16% ship (natural random sample)	67% ship (ship-enriched subset)
Best classical model	Random Forest, 93%	Random Forest, 91.9%
TL-CNN	90%	95.3%
SIM-CNN	94%	88.0%
Raw-pixel baseline	full 256×256×3 image	shrunk to 32×32×3 (speed)
Random Forest size	10 trees of depth 5 (report) / 70 trees (poster)	70 trees
Augmentation for TL-CNN	yes	no (features cached)
Framework / hardware	Keras on Azure cloud GPUs	PyTorch on a laptop CPU

What agrees

- Random Forest is the best classical model, at a similar accuracy.
- Feature engineering generally beats raw pixels.
- Naive Bayes collapses with the colour histogram and does better on texture alone.
- Both CNN architectures have exactly the paper's parameter counts.

What differs

- **The CNN ranking is reversed**: the paper's from-scratch CNN beat transfer learning; ours didn't. The likeliest reason is that we had about a quarter of their training data.
- **Absolute numbers aren't directly comparable**: different images, a different class balance, and different baselines (their "always no ship" baseline was about 84%; our "always ship" baseline is 67%).

Honest limitations

- A small, ship-enriched subset rather than a random sample of the full data.
- Each model trained once with one random seed — differences of about a point are within noise.
- No hyper-parameter search; we used the paper's settings and library defaults.
- SIM-CNN hadn't fully converged at 30 epochs.
- We classify whole images; we don't locate the ships.

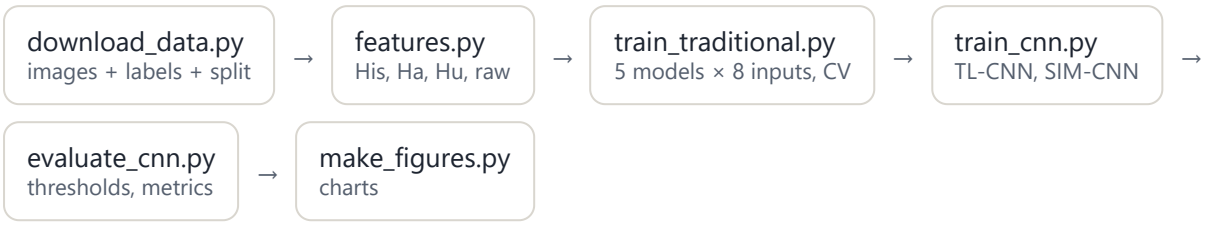
Future work

- Train on many more images, sampled uniformly from the full dataset.
- Fine-tune the later DenseNet layers instead of freezing everything.

- Train SIM-CNN longer, add batch normalisation, tune hyper-parameters.
- Move to segmentation (e.g. a U-Net) to outline each ship, as in the original competition.

15 The code, file by file

Everything lives in the `src/` folder. Each file does one stage, and `run_all.py` runs them in order.



File	What it does	Key pieces to know
`config.py`	Every setting in one place: folders, random seed (229), image size (256), split ratio (7:3:5), epochs (30), batch size (64), learning rate (0.001), thresholds (0.1–0.9).	Change a number here and it changes everywhere.
`download_data.py`	Downloads the official label CSV and the image subset from Kaggle, resizes images to 256×256, turns masks into yes/no labels, makes the stratified split, writes `data/labels.csv`.	`build_labels`, `add_subset_zip`, `add_random_images` (with retry on rate limits), `split`
`data.py`	Loads all images of a split into one NumPy array and caches it on disk so later runs are fast.	`load_split`, `read_image`
`features.py`	Computes the four feature types for every image (in parallel across CPU cores) and caches them.	`color_histogram`, `haralick`, `hu_moments`, `raw_pixels`, `combine`
`train_traditional.py`	Trains the 5 classical models on all 8 inputs (40 experiments), saves the accuracy table, runs 10-fold cross-validation and the box plot, saves the Random Forest for the demo.	`make_models`, `pipeline` (StandardScaler + model)
`models.py`	Defines the networks. Running it prints their parameter counts.	`SimCNN`, `TLHead`, `DenseNetBackbone`
`train_cnn.py`	The training loop: augmentation, Adam, learning-rate decay, best-checkpoint saving, DenseNet feature caching.	`augment`, `run_epoch`, `densenet_features`
`evaluate_cnn.py`	Threshold scan on dev, final test metrics, confusion matrices and training-curve plots.	`predict_proba`, `metrics`
`make_figures.py`	Sample images, feature illustration, class-balance and model-comparison charts.	
`demo.py`	Live demo: runs all three best models on random test images or your own image.	`predict`

Libraries used

- **NumPy** — fast arrays of numbers (our images are NumPy arrays).

- **pandas** — tables (the label CSV).
- **OpenCV** (`cv2`) — reading, resizing and colour-converting images; colour histograms; Hu moments.
- **mahotas** — Haralick texture features.
- **scikit-learn** — the five classical models, scaling, splitting, cross-validation, metrics.
- **PyTorch** and **torchvision** — building and training the CNNs, and the pretrained DenseNet-169.
- **matplotlib** — all the charts.
- **kaggle** — the official Kaggle API client for downloading data.

Reproducibility

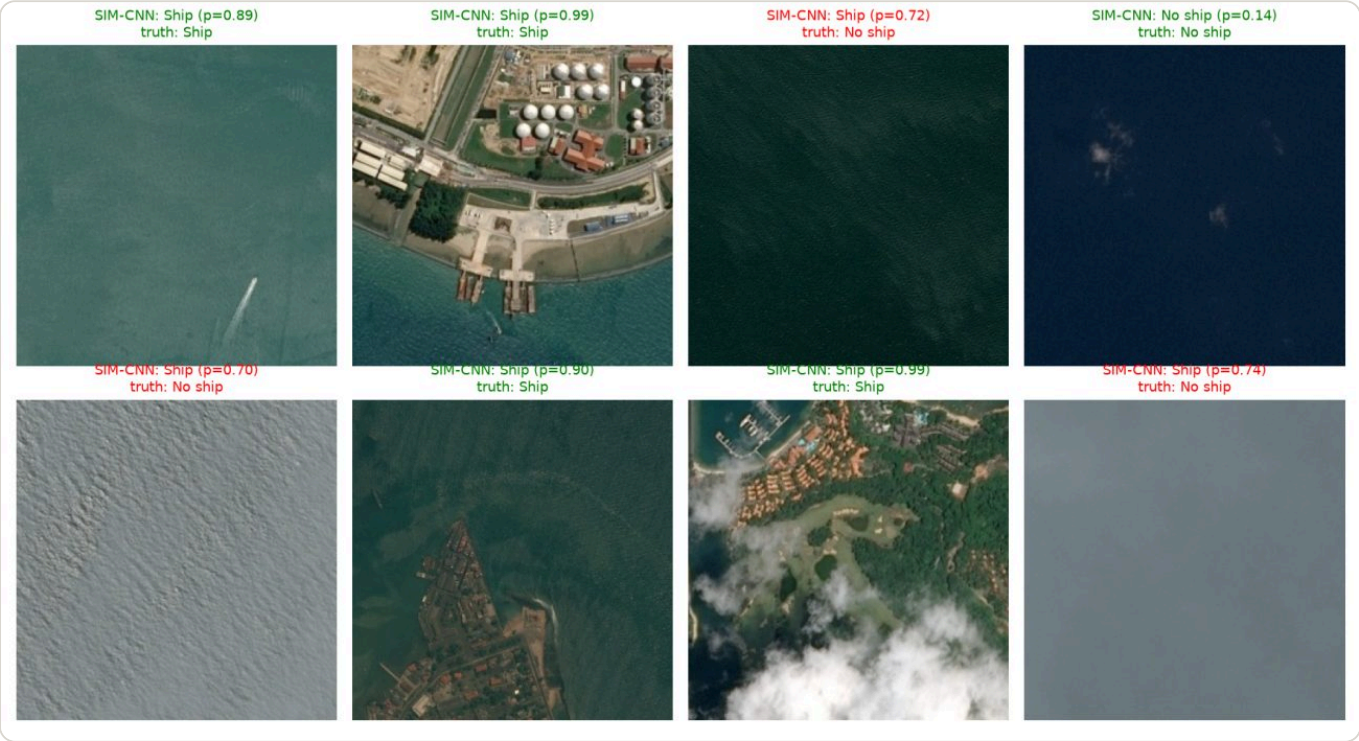
Random seeds are fixed, the exact image list and split are saved in `data/labels.csv`, package names are in `requirements.txt`, and the README gives every command. Neural-network results can still differ very slightly between computers.

16 The live demo

From the project folder, with the Python environment active:

```
python -m src.demo                      # 8 random test images (half ship, half not)
python -m src.demo --n 12 --seed 7      # a different random selection
python -m src.demo --image photo.jpg    # any image file of your own
python -m src.models                    # print both networks' parameter counts
python run_all.py                       # re-run the whole experiment
```

The demo prints a table of P(ship) from Random Forest, TL-CNN and SIM-CNN for each image and opens a picture grid with SIM-CNN's verdict on each (green title = correct, red = wrong).



Example demo output on unseen test images.

VIVA TIP

While the demo runs, point out that these images come from the test set, which no model saw during training. Pick a mistake and explain a likely cause (a tiny ship, a cloud, a dock that looks like a ship). Point out where the three models disagree. Test the demo on the presentation laptop the day before.

17 Test yourself

Try to answer each question out loud before opening it.

1. In one sentence, what does our project do?

It classifies satellite images as "ship" or "no ship", comparing classical ML models on hand-engineered features with two CNNs, reproducing a Stanford CS229 project on a subset of the Kaggle Airbus data.

2. Where do the labels come from, and how do you get yes/no from them?

From the official competition CSV of run-length-encoded ship masks, one row per ship. An image with at least one non-empty mask is "ship"; otherwise "no ship".

3. Why is accuracy alone misleading here? What's the baseline?

Because the classes are imbalanced: 67.2% of test images have ships, so always answering "ship" already scores 67.2%. That's the majority-class baseline; we also report precision, recall and specificity.

4. What is the dev set for?

Making decisions — choosing the threshold, keeping the best epoch's weights, deciding when to lower the learning rate — so the test set stays untouched for an honest final score.

5. Explain the three hand-made features.

Colour histogram: counts pixels in 512 HSV colour bins (colour). Haralick: 13 statistics of the grey-level co-occurrence matrix (texture). Hu moments: 7 shape numbers that don't change under shifting, scaling or rotation (shape).

6. Why standardise features, and what's the trap?

So features with large numeric ranges don't dominate distance-based models. The trap is computing the mean and standard deviation on test data too; we fit the scaler on training data only, inside a pipeline.

7. Why did Random Forest do best among classical models?

It handles non-linear boundaries, averages many de-correlated trees (bagging + random feature subsets) to reduce variance, is scale-invariant, and ignores useless features like empty histogram bins.

8. Why did Naive Bayes fail with the colour histogram?

It assumes features are independent and Gaussian within each class; histogram bins are strongly correlated and mostly zero, so multiplying 512 badly estimated probabilities gives over-confident wrong answers.

9. What does a convolution layer do, and why is it better than a fully connected layer for images?

It slides small learned filters over the image to detect local patterns. Weight sharing means the same detector works everywhere with very few parameters; a fully connected layer would need millions of weights and wouldn't recognise a pattern at a new position.

10. Compute the parameters of SIM-CNN's first conv layer.

$(3 \times 3 \times 3 + 1) \times 16 = 28 \times 16 = 448$.

11. What is transfer learning and what did we freeze?

Reusing a network trained on another task. We froze all of ImageNet-pretrained DenseNet-169 and trained only a small head (216,577 parameters) on its 1,664 pooled features.

12. What is overfitting, and where did you see it?

When a model fits the training data too specifically and does worse on new data. TL-CNN's training accuracy kept rising to ~99% while its dev loss rose after epoch 5; checkpointing kept the epoch-5 weights.

13. Why scan the threshold instead of using 0.5?

0.5 isn't necessarily optimal, especially with imbalanced classes. We chose the threshold with the best dev accuracy (0.5 for TL-CNN, 0.6 for SIM-CNN) and then applied it to test.

14. Why did TL-CNN beat SIM-CNN when the paper found the opposite?

Most likely data size: 1,786 training images vs 7,000. A 4.3M-parameter network from scratch needs much more data than a small head on pretrained features; SIM-CNN also hadn't converged after 30 epochs. We didn't run an experiment to prove this.

15. What is binary cross-entropy, and what's the loss if the true label is 1 and the model predicts 0.9?

loss = $-[y \cdot \ln p + (1-y) \cdot \ln(1-p)]$. With $y = 1$, $p = 0.9$: $-\ln 0.9 \approx 0.105$.

16. What does an AUC of 0.982 mean?

If you pick a random ship image and a random empty image, there's a 98.2% chance the model gives the ship image a higher P(ship).

17. Name three limitations of your project.

Small ship-enriched subset (not directly comparable to the paper); one training run per model with no hyper-parameter search; SIM-CNN not fully converged; image-level classification only, no localisation.

18. What does 10-fold cross-validation tell you that the test score doesn't?

How stable a model is across different data splits: the mean and spread of 10 scores rather than one number. Random Forest had the highest mean (92.4%) with a small spread (±1.7%).

18 Glossary

Accuracy
Fraction of predictions that are correct.

Activation function
A non-linear function applied to a neuron's output (ReLU, sigmoid).

Adam
An optimiser: gradient descent with momentum and a per-parameter adaptive step size.

Augmentation
Randomly flipping, rotating, shifting, zooming or brightening training images to create variety and reduce overfitting.

AUC (ROC-AUC)
Area under the ROC curve; the chance a random positive is scored above a random negative.

Backbone
The pretrained part of a network used as a feature extractor (our DenseNet-169).

Backpropagation
Computing all gradients by applying the chain rule backwards through the network.

Bagging
Bootstrap aggregating: train models on random resamples of the data and combine their votes.

Baseline
A trivial reference result to beat; here, always predicting the majority class (67.2%).

Batch / mini-batch
A small group of examples (64 for us) used for one gradient step.

Batch normalisation
Rescaling a layer's inputs within each batch to stabilise training.

Bias (parameter)
A constant added in a neuron before the activation.

Binary classification
Assigning each example to one of exactly two classes.

Binary cross-entropy
The loss for yes/no problems with probability outputs.

Bootstrap sample
A random sample drawn with replacement, the same size as the original.

Channel
One layer of numbers in an image or feature-map stack (R, G, B, or one filter's output).

Checkpoint
Saved model weights; we keep the one with the lowest dev loss.

Class
One of the categories being predicted (Ship, No ship).

Confusion matrix
The 2×2 table of TN, FP, FN, TP.

Convolution
Sliding a small filter across an image and computing a weighted sum at each position.

Covariance matrix
A table of how pairs of features vary together.

Cross-validation (k-fold)
Rotating which part of the data is held out, to get an average and spread of scores.

Curse of dimensionality
Distances become less meaningful as the number of features grows.

Data leakage
Test-set information influencing training, making results look better than they are.

Decision boundary
The surface in feature space that separates predicted classes.

Decision threshold
The probability above which we predict "ship".

DenseNet
A CNN in which each layer receives all earlier layers' outputs within a block.

Dev (validation) set
Data used for decisions during development, separate from training and test data.

Dropout
Randomly switching off neurons during training to reduce overfitting.

Early stopping
Keeping the weights from the epoch where dev performance was best.

Ensemble
A combination of several models whose predictions are merged.

Epoch
One complete pass over the training data.

Feature
A number describing an example that the model uses as input.

Feature engineering
Designing features by hand.

Feature map
The output grid of one convolutional filter.

Filter (kernel)
A small learned grid of weights used in convolution.

Fine-tuning
Continuing to train some or all pretrained layers on the new task.

Flatten
Turning a stack of feature maps into one long vector.

Freezing
Keeping some layers' weights fixed during training.

Fully connected (dense) layer
A layer where every input connects to every neuron.

Generalisation
Performing well on new, unseen examples.

GLCM
Grey-level co-occurrence matrix: counts of neighbouring brightness pairs.

Gradient
How much the loss changes when each parameter changes slightly.

Gradient descent
Repeatedly moving parameters a small step downhill on the loss.

Haralick features
Texture statistics computed from a GLCM.

Histogram
Counts of values falling into each of a set of bins.

HSV
Hue, Saturation, Value — a colour representation separating colour from brightness.

Hu moments
Seven shape descriptors unchanged by shifting, scaling and rotation.

Hyper-parameter
A setting chosen by people rather than learned (learning rate, number of trees, k).

Imbalanced data
One class is much more common than the other.

ImageNet
A dataset of about 1.2 million photos in 1,000 categories, used to pretrain networks.

Kernel trick
Letting an SVM act as if data were mapped to a higher dimension, without computing the mapping.

Label
The correct answer for an example.

Learning rate
The size of each gradient-descent step.

Logit
A network's raw output before the sigmoid.

Loss
A number measuring how wrong the model's predictions are.

Max-pooling
Keeping the largest value in each small block to shrink a feature map.

Model
A function with learnable parameters that maps features to predictions.

Neuron
Weighted sum of inputs plus bias, passed through an activation.

Optimiser
The algorithm that updates parameters from gradients (Adam).

Overfitting
Fitting the training data too closely and doing worse on new data.

Padding
Adding a border of zeros so convolution keeps the image size.

Parameter (weight)
A number inside a model that training adjusts.

Pipeline
A chain of processing steps (scaler then model) fitted together.

Pixel
One small square of an image, stored as numbers.

Precision
TP / (TP + FP): how often a "ship" prediction is right.

Rate limit
A cap on how many requests a server accepts in a period.

Recall
TP / (TP + FN): what fraction of real ships were found.

ReLU
max(0, z), the standard hidden-layer activation.

RGB
Red, Green, Blue — how screens store colour, 0–255 each.

Run-length encoding
Compressing a mask as (start, length) pairs.

Seed
A fixed starting number for random generators so results are repeatable.

Segmentation
Labelling exactly which pixels belong to each object.

Shortcut learning
A model succeeding by using an easier, correlated cue (e.g. coastline) instead of the intended one.

Sigmoid
1/(1+e^(−z)), squashing a number into 0–1.

Specificity
TN / (TN + FP): what fraction of empty images were correctly called empty.

Standardisation
Subtracting the mean and dividing by the standard deviation of a feature.

Stratified split
A split that keeps the class proportions equal in every part.

Supervised learning
Learning from examples that have labels.

Support vectors
The training points closest to an SVM's boundary, which define it.

Test set
Data used once, at the end, for the final unbiased score.

Training set
Data the model learns from.

Transfer learning
Reusing a model trained on one task for another.

Underfitting
A model too simple to capture the real pattern.

Reference: Y. Chen, J. Zheng, Z. Zhou, “Airbus Ship Detection — Traditional v.s. Convolutional Neural Network Approach”, Stanford CS229 project, 2018. Data: Kaggle Airbus Ship Detection Challenge.